

DEPARTAMENTO DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Trabajo práctico 2: Implementacion AirMiles

Tad AirMiles

November 7, 2026 — Algoritmos y Estructuras de Datos

Grupo: Arraydecuatroposiciones

Integrante	LU	Correo electrónico
Javier M. Ramos	288/20	ramosjavier276@gmail.com
Nancy Ramos	1206/24	Ramosnancydaniela@outlook.com
Elian Carreño	1420/23	elianuba2004@gmail.com
Leandro Amodey	618/24	leandroamodey@gmail.com

Facultad de Ciencias Exactas y Naturales

Universidad de Buenos Aires

Ciudad Universitaria - (Pabellón I/Planta Baja)

Intendente Güiraldes 2610 - C1428EGA

Ciudad Autónoma de Buenos Aires - Rep. Argentina

Tel/Fax: (++54 +11) 4576-3300

http://www.exactas.uba.ar

1 Trabajo Practico 2 - Justificaciones

Para la segunda parte de la materia, implementamos en codigo algunas de las funcionalidades descriptas en la especificacion de AirMiles.

Para esta implementacion, en primer lugar elejimos las estructuras de datos que cumplen con las complejidades pedidas. En esta eleccion tuvimos que decidir de que manera agrupar los datos primero (Usuarios y Promociones) y luego de que manera almacenarlos en memoria (Heap, ABB o Trie).

A continuacion daremos una exposicion de como lo pensamos inicialmente y luego como es que cada procedimiento/funcion cumple con las complejidades pedidas.

1.1 Clases

Elegimos tener 2 clases que representen nuestros objetos: Usuarios y Promociones.

En Usuarios, guardamos la informacion de millas disponibles y totales, categoria actual y datos necesarios para conocer a los transferidores exclusivos ** DUDA: hay una parte que hay un idReferidor que no entendi para que se usa **

En Promociones, guardamos el nombre de la misma, y los datos para saber que factor aplicar en funcion de si tiene tope o no.

1.2 Estructuras

Pasando a las estructuras elegidas, utilizamos 3: una Arbol Binario de Busqueda, un Heap y un Trie.

El ABB, lo utilizamos como estructura principal para guardar y recuperar Usuarios. De esta manera, en el caso promedio, que es que los IDs de los usuarios esten distribuidos de manera aleatoria, tendriamos un orden de complejidad O(log U). En el peor caso que seria siempre insertar IDs en orden, el orden de complejidad es similar al de una lista enlazada que es O(U)

El Trie, lo utilizamos como estructura para almacenar, recuperar y eliminar las promociones a partir del nombre de la misma. Esto nos permite incorporar la complejidad O(tamaño(L)) que es requerida en alguno de los procedimientos.

Por ultimo el MaxHeap, lo utilizamos como estructura paralela para los usuarios, ya que de esta manera podremos ordenarlos de acuerdo a un comparador, y obtenerlos en tiempos mas acotados a que si hubiera que hacerlo mediante un algoritmo. Esta estructura esta vinculada con los usuarios mediante Handles, de manera que los tengamos como una referencia directa y no haya que realizar una doble busqueda. (** A CHEQUEAR **)

1.3 Complejidad de los procedimientos

1.3.1 Crear Sistema

No requiere demasiada explicacion. Inicializar las estructuras, se realiza en un tiempo fijo que no depende de una entrada, por lo cual la complejidad es O(1)

1.3.2 Registrar Usuario - Registrar Referido

En este caso, la operacion mas costosa es la de insertar el usuario en el ABB. Esto como dijimos se realiza en O(log(U)) en promedio y en O(U) en el peor caso

1.3.3 Acumular Millas

1.3.4 Canjear Millas

1.3.5 Transferir Millas

1.3.6 Consular Saldo

1.3.7 Iniciar Promocion

1.3.8 Finalizar Promocion

1.3.9 Mas Acumulador - Mas Canjeador

1.3.10 Ranking Millas

1.3.11 Pares Transferidores Exclusivos